Lab 6 Git and GitHub for Software Development Management

Five short exercises

Clone a common SaaS project baseline, record a scope change, synchronize team updates, integrate a release checklist and correct local and shared mistakes. The five exercises cover every command in the five required Git pipelines.

Submission

Submit your public GitHub repository URL and one PDF report with exactly five short answers and five screenshots. Use the supplied report template.

Preparation

Have Git installed and a GitHub account ready. Use Git Bash on Windows so the commands in this handout match your terminal. Complete GitHub sign-in before starting.

Create an empty PUBLIC GitHub repository named SEG4105_Lab6 in your own account: click the + menu, New repository, enter the name, choose Public and click Create repository. Leave README, .gitignore and license initialization unchecked.

Create a folder named Lab6_Work on your computer. Open it in File Explorer, right-click its blank area and choose Open Git Bash here. Exercise 1 creates an empty init-demo example and clones the actual project into SEG4105_Lab6. After the cd command, all remaining edits and commands stay inside SEG4105_Lab6.

When creating README.md or release-checklist.md, use those exact names. In Windows Notepad, choose Save as type All files so the file does not become README.md.txt. Save file edits before running Git commands.

How to follow the handout

Text labelled Editor is typed into the named file, not into the terminal. Text labelled Terminal is run in Git Bash. Replace YOUR-USERNAME, YOUR-NAME and YOUR-GIT-EMAIL with your own values. The commands do not include a prompt symbol to copy.

Starter: https://github.com/Zwang-23/SEG4105_Lab6 on main. Guide and downloads: the lab-guide branch. Completed example: the worked-example branch. Clone main for your work, and publish your changes to your own empty public repository.

Exercise 1 Clone the shared project baseline

Repository Setup Pipeline

Management purpose: retrieve the shared baseline and connect it to your own submission repository.

1 Terminal initialize an example and clone the project

Start in Lab6_Work. init-demo is only an empty initialization example. The cloned SEG4105_Lab6 folder is the project you will edit.

```
git init init-demo
git clone https://github.com/Zwang-23/SEG4105_Lab6.git SEG4105_Lab6
cd SEG4105_Lab6
git config user.name "YOUR-NAME"
git config user.email "YOUR-GIT-EMAIL"
```

2 Terminal connect your own public repository

Rename the instructor remote to upstream, then add your own empty repository as origin. Replace YOUR-USERNAME in the second command with your GitHub username.

```
git remote rename origin upstream
git remote add origin https://github.com/YOUR-USERNAME/SEG4105_Lab6.git
git remote -v
git push -u origin main
git log --oneline -1
```

3 Editor check the starter file

Open the cloned SEG4105_Lab6 folder in your editor. README.md should already contain these three lines; leave them unchanged in this exercise.

```
# Tenant Portal
Version: v1
Scope: sign-in and dashboard.
```

Capture S1 in the terminal. Open your own GitHub repository to confirm that the inherited baseline is published and the repository is public.

Check the result

upstream points to the instructor repository; origin points to your own public repository. Continue working inside SEG4105_Lab6. You do not need to enter init-demo again.

Answer in one or two sentences

How do init and clone differ, and why does the team use a common starter project?

Take one screenshot

S1: one terminal screenshot showing successful clone output, git remote -v and git log --oneline -1. The origin URL must point to your own public repository.

Exercise 2 Record a scope change

Local Staging Pipeline

Management purpose: record change request CR01, adding CSV export to the project scope.

1 Editor update README md

Open the existing local README.md. Replace its entire contents with these four lines and save. Do not edit the GitHub webpage in this exercise.

```
# Tenant Portal
Version: v2
Scope: sign-in, dashboard, and CSV export.
Change CR01: CSV export requested by the product owner.
```

2 Terminal inspect then record the change

```
git status
git diff -- README.md
git add README.md
git commit -m "CR01: add CSV export to project scope"
git push
```

3 Terminal compare the two versions

```
git log --oneline -2
git diff HEAD~1 HEAD -- README.md
```

HEAD is the latest commit; HEAD~1 is its previous commit here. The comparison should show Version changing from v1 to v2 and CSV export being added. Capture S2 after these two commands.

Check the result

The history contains the baseline and the CR01 commit. Both versions remain retrievable even though README.md currently displays v2.

Answer in one or two sentences

How does a descriptive commit message help a manager track a scope change?

Take one screenshot

S2: one terminal screenshot showing git log --oneline -2 and the v1 to v2 diff.

Exercise 3 Pull and push a team update

Collaboration Pipeline

Management purpose: exchange an update about who will test the new feature. You will simulate a teammate by editing on GitHub.

1 Browser add the team update

On YOUR repository page, click README.md, then the pencil button Edit this file. Add this exact new line at the end of the file:

```
Team update: QA will test CSV export.
```

Click Commit changes. Enter the message "TEAM: assign CSV export testing to QA", select Commit directly to the main branch and confirm Commit changes. This changes GitHub; your local file is still the earlier copy.

2 Terminal fetch and pull

```
git fetch origin
git pull --ff-only
```

Reopen or reload the local README.md in your editor. The Team update line should now be present. Fetch downloads remote information; pull updates your current branch and file. No local edit should be made before this pull.

3 Editor add the manager update

At the end of the local README.md, add this exact line and save:

```
Manager update: CSV export testing assigned to QA.
```

4 Terminal commit and push

```
git add README.md
git commit -m "PLAN: confirm CSV export testing responsibility"
git push
```

Capture S3 with the pull and push success messages visible in one terminal screenshot. Refresh GitHub and check that both update lines are present.

Check the result

The local file received the team update from GitHub, and GitHub received the manager update from your local commit.

Answer in one or two sentences

Why should a team member pull the shared project plan before continuing work?

Take one screenshot

S3: one terminal screenshot showing the successful pull and push output. Scroll or resize the terminal so both are readable.

Exercise 4 Prepare and integrate a release checklist

Feature Lifecycle Pipeline

Management purpose: integrate a release checklist with the updated project plan.

1 Terminal create the feature branch

```
git branch feature/release-checklist
git switch feature/release-checklist
```

2 Editor create release checklist md

In the same local folder, create a NEW file named release-checklist.md. Type these four lines and save:

```
# Release checklist
- Test sign-in.
- Test CSV export.
- Release manager checks test results.
```

3 Terminal store and restore the unfinished draft

```
git stash push -u -m "Draft release checklist"
git stash pop
git add release-checklist.md
git commit -m "FEATURE: add release checklist"
```

The new file disappears while stashed and returns after pop. The -u option includes the new, untracked file.

4 Update main then rebase the unpublished feature

```
git switch main
```

Editor: append "Release owner: project manager." to README.md and save.

```
git add README.md
git commit -m "PLAN: name the release owner"
git switch feature/release-checklist
git rebase main
```

Rebase should succeed: main changed README.md while the feature added a different file.

5 Terminal integrate and publish

```
git switch main
git merge --no-ff --no-edit feature/release-checklist
git push
git log --graph --oneline --decorate -5
cat release-checklist.md
```

Capture S4. The checklist describes planned checks, not executed test results.

Check the result

main contains the release owner and the rebased checklist, with a merge commit recording integration.

Answer in one or two sentences

How do branch, rebase and merge help a manager integrate a feature?

Take one screenshot

S4: one terminal screenshot showing successful rebase output, the final branch graph and release-checklist.md content.

Exercise 5 Inspect and correct project records

Correction Pipeline

Management purpose: inspect responsibility for changes, undo local edits and correct a shared project record.

1 Terminal inspect the project history

```
git log --oneline -3
git blame -L 1,3 README.md
```

log lists commits; blame identifies which commit last changed each displayed line. It records change attribution, not personal fault.

2 Editor and terminal discard an uncommitted edit

Append this exact line to local README.md and save. Do not stage or commit it:

```
TEMP: accidental edit for Lab 6.
```

```
git diff -- README.md
git restore README.md
```

The diff shows +TEMP. After restore, reopen README.md: the TEMP line is gone and earlier project records remain.

3 Editor and terminal reopen a local draft commit

Append this exact line to README.md and save:

```
Release decision: skip all tests.
```

```
git add README.md
git commit -m "LOCAL: draft release decision"
git reset --soft HEAD~1
git diff --staged -- README.md
```

Do not push the LOCAL commit. Soft reset moves the branch back one commit and retains the change in the staging area. The staged diff must still show the added Release decision line.

4 Terminal reverse a shared error

Reuse that staged change to publish the deliberate error, then immediately reverse it. No additional file edit is needed.

```
git commit -m "ERROR: skip release tests"
git push
git revert --no-edit HEAD
git push
git log --oneline -2
git status --short
```

The final log must show an ERROR commit and a new Revert commit. Reopen README.md: the skip-all-tests line is gone. Empty short-status output means the working folder is clean. Capture S5 with the earlier staged diff and the final log and status visible.

Check the result

restore discarded an uncommitted edit; reset --soft reopened an unpushed draft; revert recorded the correction to the pushed error. The shared history retains the audit trail.

Answer in one or two sentences

Why use revert for a shared mistake, but restore or reset for local corrections?

Take one screenshot

S5: one terminal screenshot showing the staged diff after soft reset, the ERROR and Revert commits, and the final clean status.